Software Test Plan (STP) - Chat Application

Project: Chat Application (Socket Programming)
Version: 1.0
Authors: Team 8
Date: 2 October 2026
Status: Draft / For Validation Planning

1. Introduction

Purpose: This document defines the test plan for the Team 8 Chat Application v1.0. It outlines the testing objectives, scope, strategy, resources, schedule, responsibilities, security validation, traceability, and completion criteria.

Scope: Testing covers user registration/login, JWT/session authentication, WebSocket connection, real-time message send/receive, presence, shared-room joining, typing indicators, message history, profile updates, security requirements, performance, usability, reliability, and maintainability.

Out of Scope: Multi-room/private one-to-one chat, voice/video calling, file/media sharing beyond text, mobile push notifications, end-to-end encryption, third-party identity-provider login, and dedicated hardware interfaces are outside the current SRS scope.

Quality Objective: Demonstrate through verification and validation that the implemented system satisfies the measurable SRS requirements and provides acceptable functional, security, performance, usability, reliability, and maintainability behavior.

Roles and Responsibilities: Testing/CI is owned by Aryan Shashank Musale; API review by Annanya Sharma; overall design/UI and real-time messaging by Kaushik Krishna Dwarakanath; database responsibility by Kachhadiya Darshil Vipulkumar. All members participate in validation and defect triage.

References:

- Team 8, Software Requirements Specification (SRS) – Chat Application, Version 1.0.
- Team 8, Software Architecture Document (SAD) – Chat Application, Version 1.0.
- IEEE 829, Standard for Software and System Test Documentation (basis for this test plan structure).

Definitions and Acronyms:

- STP – Software Test Plan; SRS – Software Requirements Specification; SAD – Software Architecture Document.

- RTM – Requirements Traceability Matrix; TC – Test Case; FR / NFR / SEC – Functional, Non-Functional and Security requirement IDs from the SRS.
- JWT – JSON Web Token used for session authentication; REST – Representational State Transfer API.
- WebSocket / Socket.IO – persistent bidirectional connection used for real-time messaging.
- TLS – Transport Layer Security; HTTPS – HTTP over TLS; WSS – WebSocket over TLS.
- XSS – Cross-Site Scripting; CI – Continuous Integration.
- Critical defect – a defect that blocks a core feature (login, room join, messaging) or exposes a security vulnerability, with no workaround.

2. Test Items

- Authentication and registration module
- REST API for login, registration, and profile update
- WebSocket/Socket.IO gateway
- Shared chat room and room membership
- Real-time messaging and message broadcast
- Presence and active-user list
- Typing indicator module
- Message history and MongoDB persistence
- Profile management
- Security controls and logging
- Application performance, reliability, usability, and maintainability

3. Features to be Tested

Features mapped to SRS requirement IDs:

Requirement IDs	Feature / Validation
FR-001–FR-004	Registration, login, password hashing, JWT/session authentication
FR-005–FR-007	Persistent socket connection, broadcast, message delivery within 1 second
FR-008, FR-014	Online/offline presence and active-user list
FR-009, FR-012	Typing event and typing-indicator display/clear

FR-010, FR-013	Message persistence and retrieval of last 50 messages
FR-011, FR-015	Shared-room joining and profile update
NFR-001–NFR-005	Performance, availability, usability, reliability, maintainability
SEC-001–SEC-006	Authentication, authorization, input validation, XSS, security logging, and transport security (HTTPS/WSS)

4. Features Not to be Tested

- Multi-room or private one-to-one chat
- Voice/video calling
- File or media sharing beyond text
- Mobile push notifications
- End-to-end encryption
- Third-party Google/Facebook SSO
- Dedicated hardware interfaces
- Internal behavior of third-party libraries such as Socket.IO or MongoDB beyond their use through the application interfaces

5. Test Approach / Strategy

Overview: The plan follows the course's verification and validation approach. Verification uses static reviews of requirements, architecture, design, and code. Validation uses dynamic execution of functional, security, and non-functional tests.

Test Levels:

- Unit testing: individual authentication, validation, message, typing, and data-access modules.
- Integration testing: REST API, socket gateway, database, authentication, and service interactions.
- System testing: end-to-end chat scenarios through the browser client and backend.
- Acceptance testing: representative user tasks such as registration, login, joining the room, and sending a message.

Test Types:

- Functional testing: verify all SRS functional requirements.
- Positive and negative testing: verify valid flows and rejection of invalid credentials, invalid/expired tokens, invalid message input, and unauthorized socket access.
- Regression testing: rerun relevant tests after changes to authentication, socket events, message history, or profile modules.
- Performance testing: measure message delivery under the SRS target of 50 concurrent users and verify the 1-second delivery requirement.
- Usability testing: task-based walkthrough for registration and sending the first message.
- Reliability testing: network-drop and socket auto-reconnect scenarios.
- Availability testing: run the server continuously over a defined monitoring window, poll a health-check endpoint at fixed intervals, and compare measured uptime against the NFR-002 availability target.
- Security testing: authentication/authorization checks, input validation, XSS checks, logging checks, HTTPS/WSS transport checks, fuzzing where appropriate, and static-analysis review.

Integration Strategy: Begin with testing individual modules, then integrate dependent modules and finally perform system-level testing. Positive and negative tests are applied at each relevant level.

Entry Criteria: Build is available; required modules are integrated; test environment and database are available; test data is prepared; requirements and design references are stable enough for the planned iteration.

Exit Criteria: All planned test cases for the iteration are executed; no unresolved critical defects remain; applicable acceptance criteria are satisfied; test results and requirement traceability are updated; major security validation activities are completed.

Bug Triage: Defects are classified as Critical, Serious, Medium, or Low based on impact. Critical defects block acceptance until resolved; lower-priority defects are reviewed against the iteration schedule and scope.

5.1 Security Validation

- SEC-001: verify passwords are never stored in plaintext through code/configuration review.
- SEC-002: verify a valid session token is required before joining the room.
- SEC-003: test validation and sanitization of user-supplied message content using negative inputs.

- SEC-004: verify expired/invalid tokens are rejected on both REST and socket endpoints.
- SEC-005: test that XSS payloads are escaped before rendering in the client.
- SEC-006: verify failed login and unauthorized socket connection attempts are logged.
- Transport security: verify all REST traffic is served over HTTPS and socket connections use WSS (TLS 1.2 or higher) with a valid certificate; plain HTTP requests are redirected or rejected and unencrypted ws:// connections are refused; JWTs are never sent over an unencrypted channel.
- Use security validation methods identified in the course material: review, penetration/security testing, fuzzing where appropriate, and static analysis.

6. Test Environment

Hardware: Standard developer/test laptops and client devices capable of running a modern browser. No dedicated hardware is required by the current SRS.

Software: Browser client; Node.js + Express + Socket.IO or Python + Flask-SocketIO; MongoDB; operating-system environment suitable for the selected stack.

Tools: Jest/Mocha/Pytest for automated tests as supported by the project proposal; Postman for REST API testing; GitHub Actions for CI; SonarCloud for static code analysis; browser developer tools for client/socket inspection and TLS certificate/protocol checks; a load-testing tool may be used for the 50-concurrent-user performance test.

Test Data: valid and invalid usernames/emails/passwords, duplicate registration data, valid/invalid/expired JWTs, normal and malformed message content, XSS payloads, multiple user sessions, typing events, message history records, simulated network interruption, and plain HTTP / ws:// (non-TLS) connection attempts.

7. Test Schedule

Sprint	Dates	Planned Test Activity	Evidence
Sprint 1	21 Sep – 04 Oct 2026	Requirements review, login/registration test planning	Reviewed requirements + initial test cases
Sprint 2	05 Oct – 18 Oct 2026	Unit tests for authentication and socket setup	Unit test results
Sprint 3	19 Oct – 01 Nov 2026	Integration tests for messaging and presence	Integration test results + PR evidence
Sprint 4	02 Nov – 15 Nov 2026	System tests for typing indicators and message history	System test evidence
Sprint 5	16 Nov – 29 Nov 2026	Security validation, profile testing, performance/load testing	Security + coverage + performance results
Sprint 6	30 Nov – 13 Dec 2026	System validation, regression, acceptance and final reporting	Final test summary + validation evidence

Key milestones:

- 02 Oct 2026 – STP v1.0 baselined (this document).
- 18 Oct 2026 – Unit tests for authentication and socket setup complete; CI pipeline running.
- 01 Nov 2026 – Integration testing of messaging and presence complete.
- 15 Nov 2026 – System testing of typing indicators and message history complete.
- 29 Nov 2026 – Security validation (including HTTPS/WSS), availability and 50-user performance tests complete.
- 13 Dec 2026 – Regression and acceptance testing complete; Final Test Summary Report submitted.

8. Test Deliverables

- Test Plan (this document)
- Test cases and test data
- Automated unit/integration test scripts
- Test execution logs
- Defect reports and triage records
- Security validation evidence
- Performance/load test results
- Requirements traceability updates
- Final Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA / Testing & CI Lead	Aryan Shashank Musale	Coordinate test planning, execution, CI evidence, defect triage and test reporting.
API / Security Review	Annanya Sharma	Review authentication/API behavior and support security validation.
Dev / Team Lead	Kaushik Krishna Dwarakanath	Support real-time messaging tests, architecture/design validation and defect resolution.

Database / Test Support	Kachhadiya Darshil Vipulkumar	Support message-history/database validation and test data.
Team	All 4 members	Execute reviews, tests, defect fixes and final validation.

10. Risks and Mitigation

Risk	Mitigation
Stable build is delayed	Use early smoke builds and prioritize test-case design before final integration.
Socket/network environment instability	Maintain a reproducible local test environment and use controlled network-drop tests.
50-user load cannot be reproduced reliably	Use a controlled load-test environment and record the actual achieved concurrency.
Security defects discovered late	Run security validation throughout the SDLC and repeat it after authentication/input-handling changes.
Database/message-history issues	Use dedicated persistence tests and verify required sender, room, timestamp and last-50-message behavior.
Tool/automation issues	Keep critical functional tests executable manually while maintaining automated tests for repeatability.

11. Assumptions & Dependencies

- Users have a stable internet connection during normal chat sessions.
- The test environment can support the planned 50 concurrent socket connections.
- MongoDB and the selected backend runtime are available.
- Socket.IO/Flask-SocketIO and database drivers remain available for the implementation.
- GitHub Actions and SonarCloud are available for CI/static analysis where configured.
- Testers have access to the current SRS and architecture/design specification.

12. Suspension & Resumption Criteria

Suspend testing if:

- The test environment or MongoDB database is unavailable for more than 4 hours during a scheduled test window.
- A blocking defect in the build prevents execution of more than 30% of the test cases planned for the current sprint.

- The smoke tests for login (TC-AUTH-03) or socket connection (TC-MSG-01) fail, since all messaging, presence, typing and history tests depend on them.
- 3 or more Critical defects are open at the same time on the current build.

Resume testing if:

- The blocking defect is fixed and verified on a new build, and the smoke tests TC-AUTH-03, TC-MSG-01 and TC-ROOM-01 pass.
- The test environment and database are restored and remain available for at least 1 hour of stable smoke testing.
- Required test data and configuration are restored, and the number of open Critical defects is below 3.
- On resumption, all test cases blocked by the suspension are re-executed, followed by regression of the affected modules.

13. Test Case Management & Traceability

The RTM in the SRS maps requirements to design elements and validation evidence. Test cases use requirement-linked IDs so coverage can be measured.

Requirement	Feature	Test Case(s)
FR-001	Registration	TC-AUTH-01, TC-AUTH-02
FR-002	Login	TC-AUTH-03, TC-AUTH-04
FR-003	Password hashing	TC-AUTH-06
FR-004	JWT/socket authentication	TC-AUTH-05, TC-SEC-02
FR-005	Persistent socket connection	TC-MSG-01
FR-006	Message broadcast	TC-MSG-02
FR-007 / NFR-001	1-second delivery / 50 users	TC-PERF-01
FR-008 / FR-014	Presence and active users	TC-PRES-01, TC-PRES-02
FR-009 / FR-012	Typing indicator	TC-TYPE-01
FR-010 / FR-013	Message persistence/history	TC-HIST-01, TC-HIST-02
FR-011	Join shared room	TC-ROOM-01
FR-015	Profile update	TC-PROF-01
NFR-002	Availability	TC-AVAIL-01
NFR-003	Usability	TC-UX-01
NFR-004	Reconnect	TC-REL-01
NFR-005	Maintainability	TC-MAINT-01
SEC-001–SEC-006	Security requirements	TC-SEC-01 to TC-SEC-06
SEC-002, SEC-004 (transport)	HTTPS/WSS (TLS) enforcement	TC-SEC-07

14. Test Metrics & Reporting

Metrics collected:

- Number of test cases planned, created, executed and passed/failed.
- Percentage of planned test cases executed.
- Number and severity of defects found.
- Defect density and defect aging where meaningful for the project.
- Requirement coverage through the RTM.
- Message delivery response time and achieved concurrent-user load.
- Unit-test/code coverage available from the selected test framework and CI.

Reports:

- Sprint/iteration test execution status.
- Defect and triage summary.
- Security validation summary.
- Performance/load test report.
- Final Test Summary Report.

15. Approvals

Role	Name	Signature / Date
QA / Testing & CI Lead	Aryan Shashank Musale	
Dev / Team Lead	Kaushik Krishna Dwarakanath	
Course Coordinator		